SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1007	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	Sk. Parvez Ahamed	**Reg. No.:**	192411084
Date:	30/07/2026	**Duration:**	60 minutes

Code of Conduct

I _____Sk.Parvez Ahamed (192411084)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____Sk. Parvez Ahamed_____

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:
Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.

- Identify the business domain, stakeholders, existing challenges, and system objectives.

2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description
   - Scope of the System
   - Functional Requirements
   - Non-Functional Requirements
   - Use Case Descriptions
   - Data Requirements
   - External Interface Requirements
   - System Features
5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

S.No : 13

Blockchain-Based Academic Credential Verification Platform

1. Analyze System Requirements

1.1 Introduction

The Blockchain-Based Academic Credential Verification Platform is designed to provide a secure and transparent system for issuing, storing, and verifying academic certificates using blockchain technology. Traditional verification methods rely on paper documents and centralized databases, making them vulnerable to forgery, document tampering, and lengthy verification processes. The proposed platform eliminates these issues by storing certificate

hashes on a blockchain, allowing employers and institutions to verify credentials instantly while ensuring data integrity and security.

1.2 System Objectives

The primary objectives of the proposed system are:

- Issue secure digital academic certificates.
- Prevent certificate forgery and tampering.
- Enable instant certificate verification.
- Maintain transparent audit trails.
- Support multiple educational institutions.

1.3 Functional Requirements

The system should provide the following functionalities:

- User Registration and Login
- Student Profile Management
- University Registration
- Blockchain Hash Storage
- QR Code Generation
- Certificate Verification
- Verification Report Generation

- Audit Log Management

1.4 Non-Functional Requirements

The system should satisfy the following quality requirements:

- High Security
- Fast Performance
- High Reliability
- Scalability

1.5 Quality Attributes

The software architecture should support:

Scalability

Support multiple universities and millions of certificates without performance degradation.

Maintainability

The modular design allows developers to update individual services independently.

Reliability

Blockchain technology ensures data integrity and prevents unauthorized modifications.

Availability

The system should remain accessible 24×7 with minimal downtime.

Performance

Certificate verification should complete within a few seconds.

Security

Digital signatures, blockchain storage, encryption, and role-based authentication protect sensitive information.

2. Select a Suitable Software Architecture

Selected Architecture

Hybrid Architecture (Layered + Microservices Architecture)

Justification

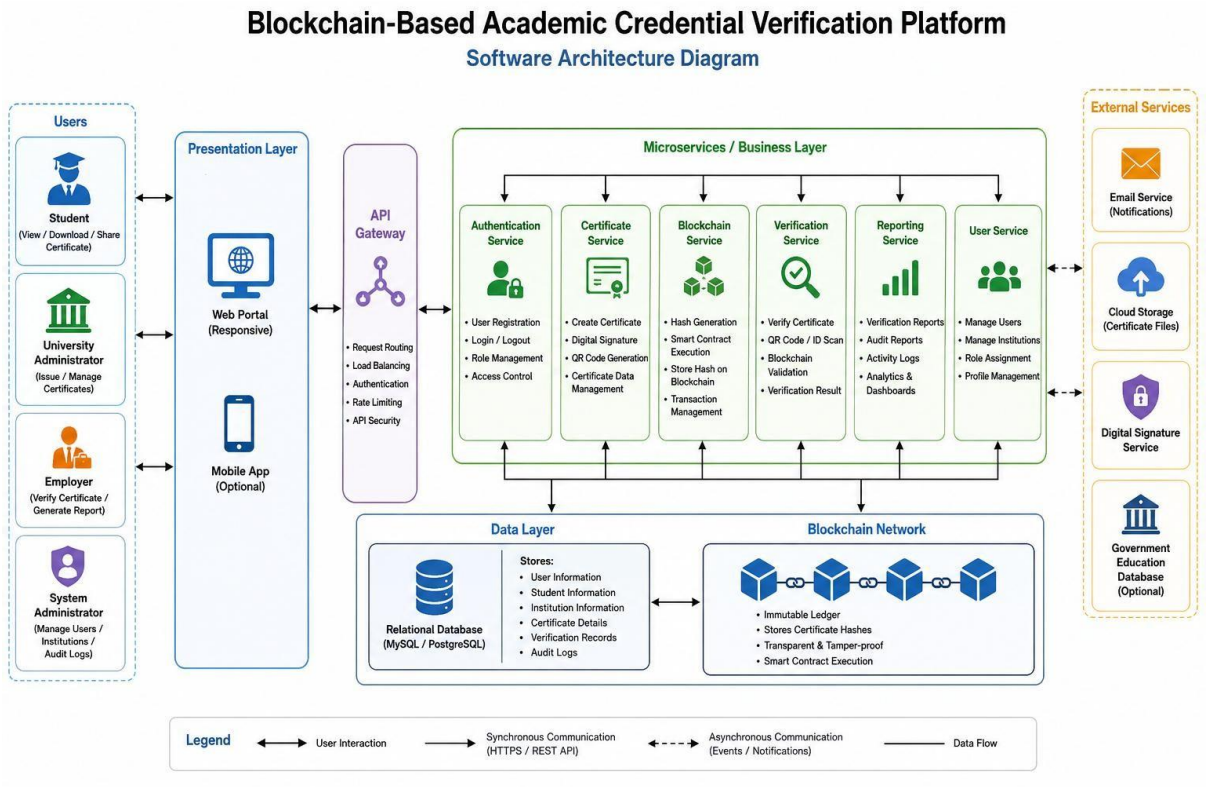
A Hybrid Architecture combines the advantages of Layered Architecture and Microservices.

The Layered Architecture separates the application into Presentation, Business Logic, Data Access, and Database layers, improving maintainability and modularity.

Microservices divide major functionalities such as authentication, certificate management, blockchain integration, verification, and reporting into independent services. Each service can be developed, deployed, and scaled independently.

This architecture is suitable because the proposed system involves multiple users, blockchain integration, external APIs, and future scalability.

3. Software Architecture Diagram



4. Explain Components and Their Interactions

4.1 Presentation Layer

The Presentation Layer provides a user-friendly interface for students, universities, employers, and administrators. Users interact with the platform through a web or mobile application.

4.2 API Gateway

The API Gateway receives all client requests and forwards them to the appropriate microservice. It also handles authentication, routing, and request validation.

4.3 Authentication Service

This service manages user registration, login, password management, and role-based authentication.

Responsibilities:

- User Login
- User Registration
- Role Management
- Access Control

4.4 Certificate Service

The Certificate Service handles academic credential management.

Responsibilities:

- Generate Certificates
- Digital Signature
- QR Code Generation
- Certificate Storage

4.5 Blockchain Service

The Blockchain Service generates certificate hashes and stores them on the blockchain.

Responsibilities:

- Hash Generation
- Smart Contract Execution
- Blockchain Transactions
- Certificate Validation

4.6 Verification Service

The Verification Service allows employers and institutions to verify certificates.

Responsibilities:

- QR Code Verification
- Certificate ID Verification
- Blockchain Comparison
- Verification Result

4.7 Reporting Service

The Reporting Service generates verification reports, audit logs, and institutional reports.

Responsibilities:

- Verification Reports
- Audit Reports
- Activity Logs

4.8 Database

The database stores user information, student records, institution details, and verification history. Only certificate hashes are stored on the blockchain, while the complete certificate information is maintained in the relational database.

4.9 System Workflow

The overall system operates as follows:

1. The university administrator logs into the platform.
2. Student information is verified.
3. A digital certificate is generated.
4. A certificate hash is created.
5. The hash is stored on the blockchain.
6. A QR code is generated.
7. The student downloads the certificate.
8. The employer scans the QR code.
9. The verification service compares blockchain records.
10. The system displays the verification result and generates a report.

5. Discuss Quality Attributes

Scalability

The use of microservices allows independent scaling of services based on demand. New institutions can be added without affecting existing users.

Security

The architecture ensures security through:

- Blockchain technology
- HTTPS communication
- Role-Based Access Control (RBAC)

Performance

Independent services reduce processing time and improve certificate verification speed.

Reliability

Blockchain provides immutable storage, while service isolation minimizes the impact of component failures.

Maintainability

Each service can be updated independently without affecting the complete application.

Availability

Cloud deployment and distributed services ensure high availability and continuous operation.

6. Justify Architectural Decisions

Architectural Rationale

The Hybrid Architecture was selected because it combines modular design with the flexibility of microservices. Layered Architecture organizes the system into logical layers, while Microservices enable independent deployment and scaling of critical services such as certificate issuance and blockchain integration.

Trade-offs

Advantages

- Easy Maintenance
- High Scalability
- Better Security
- Independent Deployment
- Faster Development
- Fault Isolation

Limitations

- Higher Infrastructure Cost
- Complex Service Communication
- Requires DevOps Support
- More Deployment Complexity

Future Enhancements

The architecture supports future improvements such as:

- AI-Based Fraud Detection
- Mobile Applications
- Biometric Authentication
- Government Database Integration

- Cloud-Native Deployment

Long-Term Maintainability

The modular architecture simplifies future upgrades and maintenance. Independent services allow new features to be added without affecting the entire system. The use of APIs and standardized communication protocols also makes integration with external educational systems easier.

Conclusion

The Blockchain-Based Academic Credential Verification Platform adopts a Hybrid Architecture (Layered + Microservices) to provide a secure, scalable, and maintainable solution for academic credential management. The architecture separates responsibilities into independent services such as authentication, certificate management, blockchain integration, verification, and reporting, ensuring better performance and flexibility. By leveraging blockchain technology, digital signatures, and QR-code-based verification, the system prevents certificate fraud, enables instant verification, and supports future enhancements, making it a reliable solution for modern educational institutions.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modeling	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation	SRS is well-structured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.
Assumptions, Constraints, and Requirement Validation	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis	/5
Functional and Non-Functional Requirement Identification	/5
Actor Identification and Use Case Modeling	/5
Software Requirement Specification (SRS) Documentation	/5
Assumptions, Constraints, and Requirement Validation	/5
TOTAL	/5